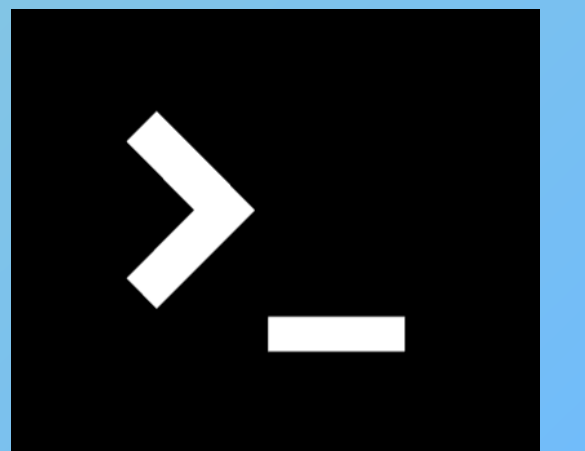


Linux y Terminal

Clase 14



◀ Índice ▶

Algunos comandos

Locate

Find

Tar

Zip

Redirección de entrada y error

Pipes

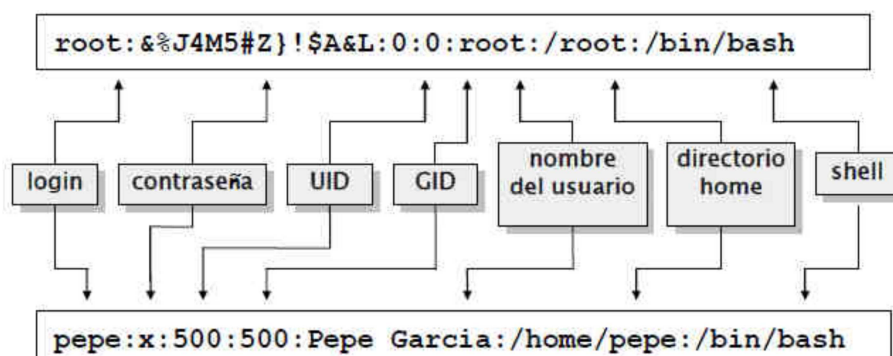
Ejecución de varios comandos a la vez y segundo plano

5 ARCHIVOS DE SISTEMA RELACIONADOS CON USUARIOS Y GRUPOS.

Las cuentas de usuario y grupo, así como las contraseñas de los mismos se almacenan en el sistema en los ficheros `/etc/passwd`, `/etc/shadow`, `/etc/group` y `/etc/gshadow`. Estos ficheros pueden ser modificados manualmente para gestionar usuarios y grupos, pero no es una práctica recomendable.

5.1 FICHERO DE USUARIOS (PASSWD).

Cuando se añade un usuario al sistema se añade una línea al fichero de usuarios `/etc/passwd`. Cada línea de este fichero contiene información sobre una cuenta de usuario, con campos separados por el símbolo dos puntos. La estructura de cada línea es la siguiente:



nombre-cuenta:contraseña:UID:GID:nombrecompleto-usuario:directorio-home-usuario:shellpor-defecto

Veamos un poco más en profundidad cada uno de los campos:

- **Nombre-cuenta:** Es el nombre de la cuenta de usuario y debe ser único en el fichero.
- **Contraseña:** Cada nombre de usuario tiene asignada una contraseña. La contraseña aparece en este campo de una forma cifrada (ininteligible e irreversible). Por razones de seguridad la mayoría de las distribuciones almacenan las contraseñas en un fichero separado `/etc/shadow`, apareciendo en este campo `/etc/passwd` una letra X.
- **UID:** Identificador del usuario. Este UID es un entero no negativo, asignándose al root el UID 0 que otorga permisos especiales en el sistema. Normalmente los valores de 0 a 99 son reservados para uso de administración. En muchas distribuciones este número para usuarios normales comienza en 500 o 1.000.
- **GID:** Identificador del grupo. Este GID es un entero no negativo. La correspondencia entre este número y el nombre del grupo se especifican en el fichero `/etc/group`.
- **Nombre-completo:** El nombre completo del usuario. Muchas distribuciones utilizan este campo para almacenar otro tipo de comentarios. Este campo puede contener espacios en blanco.
- **Directorio-home-usuario:** Indica el directorio inicial del usuario. Este directorio suele estar situado en `/home/nombre-usuario` y suele incluir ficheros que sirven para configurar y personalizar el entorno del usuario.
- **Shell-por-defecto:** El programa intérprete de comandos (Shell) que el usuario utiliza por defecto. En la mayoría de los casos este Shell suele ser `/bin/bash`, pero puede ser cualquier otro. En ocasiones se especifica aquí un programa no ejecutable como `/bin/false`, a efectos de impedir que ese usuario pueda abrir ninguna Shell al hacer login, y por lo tanto, no pueda ejecutar ningún comando en el sistema.

5.2 FICHERO DE CONTRASEÑAS DE USUARIO (SHADOW).

Como hemos indicado anteriormente, las contraseñas habitualmente suelen estar almacenadas en el fichero `/etc/shadow`. Este fichero cuenta con unos permisos mucho más restrictivos que `/etc/passwd` y solo puede ser accedido por el usuario `root`. En cada línea de este fichero se almacena la contraseña y otra información referente a un usuario.

Una línea de `/etc/shadow` suele ser algo parecido a lo siguiente:

```

backup:*:16176:0:99999:7:::
list:*:16176:0:99999:7:::
irc:*:16176:0:99999:7:::
gnats:*:16176:0:99999:7:::
nobody:*:16176:0:99999:7:::
libuuid:*:16176:0:99999:7:::
syslog:*:16176:0:99999:7:::
messagebus:*:16176:0:99999:7:::
usbmux:*:16176:0:99999:7:::
dnsmasq:*:16176:0:99999:7:::
avahi-autoipd:*:16176:0:99999:7:::
kernoops:*:16176:0:99999:7:::
rtkit:*:16176:0:99999:7:::
saned:*:16176:0:99999:7:::
whoopsie:*:16176:0:99999:7:::
speech-dispatcher:*:16176:0:99999:7:::
avahi:*:16176:0:99999:7:::
pulse:*:16176:0:99999:7:::
colord:*:16176:0:99999:7:::
hplip:*:16176:0:99999:7:::
gdm:*:16176:0:99999:7:::
debian-spamd:*:16176:0:99999:7:::
royer:$6$Vp/wSLcs$U0TeffHqG5Iqseyo4XvNXnhwcJlBHz2yXXc5qR.MdwxEK4K0UfGB0zW5HNEmxKLaDk7I1D.ZJwLBGY/:16344:0:99999:7:::
vboxadd:*:16356:0:99999:7:::
mama:$6$K/fL8WE8$Xbd7MVCbDeRr.m6P3fL4HpH34pBycU0SPHwBeTLA1HyufMhP0ApSZjoqzAZUCP3RF8RjVkv3c6fx08btLkQwm0:16371:0:99999:7:::
papa:$6$0SpBn0fZ$Y8zS1968tUR0tLby4jff7a4QyUPMW7bZpXK9NU8YzwQsnuJHLruoBQLvSPVksRsaHhC1ovFwm.4z3tDHmb8mL1:16371:0:99999:7:::
asun:$6$Q/m6XrGN$LYW3GPdJjRZxx5Z2cLm1fKp3VDaSwk0w4EFqR710.RAA.e.zR5ITbz.soLAD7LAU0cWfuKmRi99KYp1jwXx4k0:16376:0:99999:7:::
$LYW3GPdJjRZxx5Z2cLm1fKp3VDaSwk0w4EFqR710.RAA.e.zR5ITbz.soLAD7LAU0cWfuKmRi99KYp1jwXx4k0:16376:0:99999:7:::

```

Como vemos, al igual que en el fichero `/etc/passwd` cada línea cuenta con varios campos separados por el símbolo dos puntos. Cada uno de estos campos almacena lo siguiente:

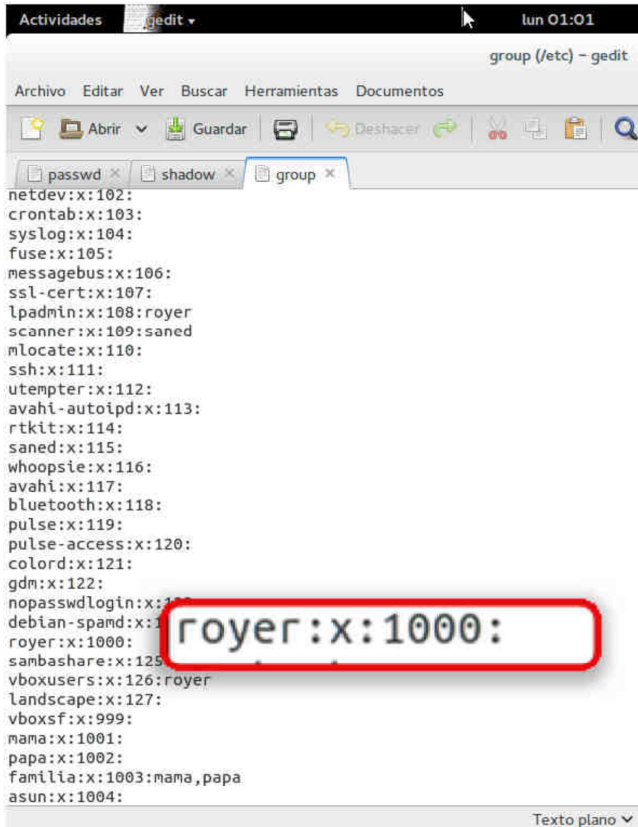
- Nombre de usuario: El nombre de usuario, tal como existe en `/etc/passwd`.
- Contraseña: La contraseña se almacena de forma encriptada, de modo que viendo lo aquí almacenado no se puede averiguar la verdadera contraseña. Si no aparece aquí ninguna contraseña (::) el usuario podrá abrir sesión sin contraseña. Un asterisco o un símbolo de exclamación (!) indican que dicho usuario está bloqueado y no puede abrir sesión en el sistema.
- Último cambio de contraseña: (16344 en nuestro ejemplo). La fecha en la que se cambió la contraseña por última vez. Esta fecha se almacena indicando el número de días que pasan desde el 1 de enero de 1970.
- Días hasta que se permita un cambio de contraseña: Número de días que deben transcurrir para que al usuario le sea permitido cambiar la contraseña.
- Días antes de que sea requerido un cambio de contraseña: Número de días desde el último cambio de contraseña antes de que la contraseña caduque y se le solicite otra al usuario.
- Días entre la expiración y la desactivación: Linux permite unos días entre la expiración de una cuenta y su desactivación. En este campo se indica este periodo de gracia.
- Fecha de expiración de la cuenta: Fecha en la que la cuenta expirará. La fecha se almacena como el número de días transcurridos desde el 1 de enero de 1970.
- Campo reservado: Este campo se reserva para futuros usos, y no se utiliza actualmente para nada. Normalmente está vacío.

En los campos relativos a número de días, un valor de -1 o de 99999 indica que esa característica está deshabilitada en el sistema. Estos valores se modifican con los comandos `usermod` y `chage`.

5.3 FICHERO DE GRUPOS (GROUP).

Las cuentas de grupo están almacenadas en el fichero `/etc/group`.

Al igual que en el fichero `/etc/passwd` cada línea contiene una serie de campos separados por el símbolo dos puntos.



```

netdev:x:102:
crontab:x:103:
syslog:x:104:
fuse:x:105:
messagebus:x:106:
ssl-cert:x:107:
lpadmin:x:108:royer
scanner:x:109:saned
mlocate:x:110:
ssh:x:111:
utempter:x:112:
avahi-autoipd:x:113:
rtkit:x:114:
saned:x:115:
whoopsie:x:116:
avahi:x:117:
bluetooth:x:118:
pulse:x:119:
pulse-access:x:120:
colord:x:121:
gdm:x:122:
nopasswdlogin:x:123:
debian-spand:x:124:
royer:x:1000:
smbshare:x:125:
vboxusers:x:126:royer
landscape:x:127:
vboxsf:x:999:
mama:x:1001:
papa:x:1002:
familia:x:1003:mama,papa
asun:x:1004:

```

Cada uno de estos campos almacena lo siguiente:

- Nombre de grupo: Cada grupo debe contar con un nombre de grupo único.
- Contraseña del grupo: Los grupos pueden contar con contraseñas, al igual que las cuentas de usuario. De la misma forma, en este campo no se almacena la contraseña del grupo, quedando para esta labor el fichero `/etc/gshadow`.
- GID: Identificador del grupo. Un número no negativo que identifica al grupo en el sistema.
- Miembros: Aquí podemos encontrar una serie de nombres de usuario, separados por comas. Todos los usuarios en esta lista son miembros del grupo. La lista puede estar vacía o contar con decenas de usuarios.

Aquí vemos un ejemplo de una línea del fichero `/etc/group`.

```
trabajadores:x: 1005: luis,pedro,carmen
```

En esta línea se almacena un grupo con nombre `trabajadores`, con GID 1005 y que cuenta con 3 usuarios como miembros del grupo (luis, pedro y carmen).

Cuando se crea un nuevo usuario ya vimos como es habitual que se cree también un grupo con su mismo nombre. En este caso, el nuevo usuario es automáticamente asignado por el sistema Linux como miembro del grupo creado con su mismo nombre. Sin embargo, el usuario no aparecerá como miembro en el fichero `/etc/group`. Este es el comportamiento por defecto para estos grupos unipersonales.

rm	Borra un fichero de manera permanente -r → Elimina recursivamente un directorio	<code>rm -r <directorio></code> <code>rm <fichero></code>
mv	Renombra o mueve un fichero de lugar	<code>mv prueba1.txt prueba2.txt</code> <code>mv prueba1.txt /home/usuario/</code>
cat	Muestra el contenido de uno o varios ficheros	<code>cat prueba1.txt</code>
file	Indica el tipo de fichero	<code>File /sbin/iptables</code>
du	Indica la ocupación de un directorio -h → Muestra la info en Gb, Mb,... -s → Muestra solo la ocupación del directorio sin sus subdirectorios	<code>du -hs /home/usuario</code>
df	Muestra información del sistema de ficheros -h → Muestra la info en Gb, Mb,...	<code>df -h</code> <code>df -hs /home/usuario</code>
head	Muestra las primeras de un fichero	<code>head /home/usuario/prueba1.txt</code> <code>head -2 listado.txt</code>
tail	Muestra las últimas líneas de un fichero	<code>tail /home/usuario/prueba1.txt</code>
touch	Crea un fichero vacío con 0 bytes	<code>touch prueba3.txt</code>
wc	Muestra el número de líneas, palabras y caracteres de un fichero -l → Muestra las líneas -w → Muestra las palabras -c → Muestra los caracteres	<code>wc fichero.txt</code> <code>wc -l</code> <code>wc -w</code> <code>wc -c</code>
more	Muestra el contenido de un fichero de forma paginada	<code>more fichero.txt</code>
less	Muestra el contenido de un fichero de forma paginada	<code>less fichero.txt</code>
sort	Ordena por orden alfabético, normalmente las líneas de un fichero, un listado, ...	<code>sort fichero.txt</code>
ln	Enlaza o crea un acceso directo a un fichero -s → crea un enlace simbólico	<code>ln -s fichero.txt enlace.lnk</code>
cut	Permite sacar campos o simplificar ficheros mucho más complejos	<code>cut -d " " -f 2,4 futbol.txt</code> Muestra la columna 2 y 4 de un fichero de texto separado por el carácter " "
grep	Permite encontrar un determinado patrón dentro de una cadena o fichero	<code>grep "Messi" futbol.txt</code>
locate	Localiza el directorio se encuentra el fichero a buscar	<code>locate firefox.sh</code>
find	Busca un fichero o directorio a partir de la ruta que le indicamos	<code>find /home/usuario -name firefox</code>
tree	Muestra el árbol de directorios de nuestro	<code>tree</code>

	<i>sistema</i>	
whereis	<i>Localiza un fichero ejecutable</i>	<i>whereis firefox</i>

Comando: ls

Definición: Muestra información sobre ficheros y directorios y si no se especifica nada se muestra el directorio actual.

Sintaxis: ls [-opciones] [argumentos]

Opciones:

-a: Nos muestra información sobre los archivos ocultos (empiezan por punto).

-h: Junto con la opción -l muestra el tamaño del fichero en la unidad de medida mayor para que se pueda entender.

-l: Muestra información en formato largo.

Nota: La información que obtenemos con la opción “-l” es la siguiente:

a) El primer carácter hace referencia al tipo de archivo que deseas:

- “-” Significa que es un fichero regular.
- “l” Indica que es un enlace simbólico.
- “d” Indica que es un directorio.
- También puede aparecer una “b”, “c” o una “p” para archivos especiales de bloques, caracteres o tuberías respectivamente.

b) Los siguientes nueve caracteres indican los permisos que los archivos tienen o no activados. Los permisos pueden ser: “r” (lectura), “w” (escritura), “x” (ejecución), “-” (no tiene activo el permiso). Hay tres grupos de permisos, agrupados en bloques de tres caracteres; el primer bloque corresponden al propietario del fichero, el segundo bloque indica los permisos del grupo al que pertenece el propietario y el tercer bloque los permisos del resto de usuarios del sistema.

c) El número siguiente es el número de enlaces duros que tiene ese directorios.

d) El siguiente nombre indica el propietario o fichero.

e) El siguiente nombre indica el grupo del fichero.

f) El siguiente número es el tamaño del fichero en bloque.

g) El siguiente número es la fecha y hora de creación/modificación del fichero.

h) El siguiente nombre es el nombre del fichero: Si es un enlace simbólico además veremos también a qué archivos y directorios que apuntan.

Argumento: Directorios o ficheros sobre los que hacer el listado.

Comando: pwd

Definición: Muestra la ruta absoluta del directorio en el que estamos.

Sintaxis: pwd

Comando: mkdir

Definición: Crea directorios.

Sintaxis: mkdir [-opciones] directorio[/s]

Opciones:

-v: Sirve para mostrarnos un mensaje por cada directorio que hace.

-p: Si creamos un subdirectorio y su padre no existe, también crea el directorio padre.

Argumentos: Direcciones relativas o absolutas del directorio/s a crear. Si vamos a crear varios directorios se separan por un espacio.

Ampliación:

mkdir FOL MME: Crea dos carpetas FOL y MME.

mkdir Redes OFI/EV2: Crea la carpeta Redes y la carpeta EV2 dentro de OFI.

mkdir Curso/Redes Curso/OFI/EV1 Curso/OFI/EV2: Sirve para ir creando una estructura completa.

Comando: cd

Definición: Cambia de directorio.

Sintaxis: cd [-opciones] [directorio/s]

Opciones: Ninguna

Directorios: Se le puede indicar por una ruta absoluta o relativa o directorios especiales como el cd .. (padre) o el directorio personal con cd ~

Comando: rmdir

Definición: Borra los directorios si están vacíos.

Sintaxis: rmdir [-opciones] directorios

Opciones:

-p: Borra los directorios y sus antecesores.

Argumentos: Directorio/s que queremos borrar.

Comando: rm

Definición: Borra ficheros y directorios.

Opciones:

-d: Borra directorios aunque no estén vacíos.

-i: Pregunta antes de borrar cada fichero o directorio.

-r: Borra los directorios, los ficheros que contengan y los subdirectorios que estén por debajo de forma recursiva.

Argumento: Borra los ficheros o directorios que se le indiquen separados por espacio. Se pueden utilizar comodines.

Comando: cp

Definición: Copia uno o varios ficheros en otros ficheros o en un directorio.

Sintaxis: cp [-opciones] ficheros destino

Opciones:

-r: Copia directorios y los que estén por debajo de él.

-i: Pregunta antes de sobrescribir.

-f: Fuerza la copia. Si el destino no existe y no se puede abrir, lo borra e intenta copiar de nuevo.

Ampliación:

cp ofi/EV1/f1.txt ofi/EV2 El primer trozo es lo que quiero copiar y el segundo es donde lo quiero copiar.

Para copiar dos o más comandos: cp ofi/EV1/f1.txt ofi/EV1.f2.txt Redes

Comando: mv

Definición: Mueve uno o varios ficheros a otro fichero o directorio. Es equivalente a copiar un archivo y borrar el original. Se utiliza para renombrar ficheros.

Sintaxis: mv [- opciones] ficheros destino

Opciones:

-u: Mueve solo si el destino no existe o si es posterior al que se va a sobrescribir.

-i: Pregunta antes de sobrescribir.

Ampliación:

mv f1.txt f3.txt Con este comando simplemente se renombra el fichero.

mv f3.txt ../EV2 Mueve el fichero.

mv f1.txt ../../Redes/f4.txt Mueve el fichero y lo renombra.

Comando: cat

Definición: Muestra el contenido de los ficheros que se le pase como argumento.

Sintaxis: cat [-opciones] fichero/s

Nota: El comando cat además de visualizar el contenido de un fichero, lo utilizaremos para crear nuevos ficheros de texto, utilizando los símbolos de redireccionamiento >, >>.

Opciones:

-n: Numera todas las líneas.

Ampliación:

cat > f1.txt Crea un fichero de texto.

cat >> f1.txt Lo que escribas se añade al fichero de texto.

Comando: file

Definición: Muestra el tipo de fichero.

Sintaxis: file fichero

Comando: du

Definición: Muestra el espacio que ocupa el fichero o directorio.

Sintaxis: du [-opciones] [argumento]

Opciones:

-b: Muestra el tamaño en bytes.

-h: Muestra el tamaño en la unidad de medida mayor que se pueda para que se pueda entender mejor.

Argumento: Se pueden pasar uno o varios ficheros o directorios.

Comando: df

Definición: Muestra el espacio libre en los dispositivos de almacenamiento y en las particiones montadas.

Sintaxis: df [-opciones]

Opciones:

-h: Tamaño.

-a: Muestra todos incluso los que tengan tamaño 0.

Comando: head

Definición: Muestra por defecto las 10 primeras filas de un documento o varios.

Sintaxis: head [-opciones] fichero/s

Opciones:

-n: Muestra las n (el número que tú elijas) primeras líneas en lugar de las primeras 10.

Ejemplos: Si queremos mostrar las 3 primeras líneas escribiremos *head -n3* o simplemente *head -3*

Comando: tail

Definición: Muestra por defecto las 10 últimas líneas de un fichero de texto.

Sintaxis y opciones: Igual que en el head.

Comando: wc

Definición: Muestra el número de líneas, palabras, caracteres y bytes de los ficheros que se le indiquen.

Sintaxis: wc [-opciones] fichero/s

Opciones:

-c: Sirve para mostrar el número de bytes.

-m: Muestra el número de caracteres.

-l: Muestra el número de líneas.

-w: Muestra el número de palabras.

Comando: more

Definición: Muestra el contenido de los ficheros de forma paginada, es decir, pantalla a pantalla. Si el contenido del fichero ocupa más de una pantalla se quedará esperando a que se pulse una tecla:

- Si pulsamos la barra espaciadora, mostrará la página siguiente.

- Si pulsamos la tecla Entrar, mostrará la siguiente línea.

- Si pulsamos la tecla Q, finaliza la ejecución.

Sintaxis: more fichero/s

Comando: less

Definición: Funciona igual que el more pero además nos permite movernos por el fichero con los cursores.

Sintaxis: less fichero/s

Comando: sort

Definición: Muestra en orden ascendente el contenido de los ficheros que se le pasa como argumento. También sirve, utilizando redireccionamiento para crear un fichero ordenado.

Sintaxis: sort [-opciones] [fichero/s]

Opciones:

-r: Ordena en sentido inverso, de mayor a menor o descendentemente.

-u: Elimina las líneas repetidas.

-c: Comprueba si el está ordenado. Si no está ordenado re muestra un mensaje indicando la primera línea que está en desorden.

-m: Mezcla ficheros que ya estén ordenados.

> Para crear un fichero.

Comando: ln

Definición: Crea un enlace al fichero o al directorio que se le especifique. Si es un directorio el enlace deberá ser simbólico.

Sintaxis: ln [-opciones] fichero enlace

Opciones:

-s: Crea un enlace simbólico (acceso directo).

Comando: cut

Definición: Muestra solo ciertas líneas verticales de los ficheros que se le pase como argumento.

Sintaxis: cut [-opciones] [ficheros]

Opciones:

- c: Muestra solo los caracteres que se le especifican.
- d: Usa el carácter que se le especifique como delimitador en vez de tabulador.
- f: Muestra los campos que se le indiquen.

Ejemplos:

echo "Esto es una prueba, 1 2 3, probando" | cut -d "," -f1 → Lo que mostraría es Esto es una prueba.

Comando: grep

Definición: Muestra las líneas de un fichero dado que coinciden con un cierto patrón.

Sintaxis: grep [-opciones] patrón [fichero/s]

Opciones:

- r: Busca un patrón de forma recursiva dentro de los ficheros de un directorio.
- n: Muestra el número de línea en el que se encuentra el patrón.
- i: Ignora las mayúsculas/minúsculas.
- v: Muestra líneas en las que no se encuentra el patrón.
- c: Escribe el número de líneas que contiene el patrón.
- w: Busca las líneas en las que aparece el patrón completo y no como parte de otra palabra.
- l: Se escriben los nombres de los ficheros que contienen líneas de patrones buscados.

Patrón: Busca líneas que contengan la palabra texto (el texto que tú quieras).

^texto: Busca líneas que empiecen por texto.

texto\$: Busca líneas que terminen por la palabra texto.

^[^texto]: Busca las líneas que no empiecen por texto.

Comando: locate

Definición: Busca archivos por su nombre dentro del sistema. Es muy rápido porque busca en una base de datos propia que se actualiza periódicamente.

Sintaxis: locate fichero

Comando: find

Definición: Busca ficheros en un árbol de directorios. Muestra y/o opera con los archivos encontrados según ciertos patrones o criterios.

Sintaxis: find [-opciones] [directorio/s] [criterios] [acción] \;

Opciones:

-L: Sigue los enlaces simbólicos si apuntan a directorios.

Directorio: El directorio a partir del cual queremos buscar.

Criterio:

-type f: Busca ficheros regulares.

-type d: Busca directorios.

-type l: Busca enlaces simbólicos.

-name nombre (de lo que busquemos): Encuentra los archivos cuyo nombre coincida con el nombre dado. Distingue entre mayúsculas y minúsculas. Si no quisiéramos que los diferenciara, en lugar de poner -name utilizaremos -iname. Se pueden combinar criterios con la opción -a (and), -o (o) y el -not (no).

Acción: -exec comando (a elegir)

Ejemplo: Busca en el directorio actual todos los ficheros con extensión .txt o .odt y que los copie al directorio personal.

find -name “*.txt” -o -name “*.odt” -exec cp {} ~ \;

Comando: tree

Definición: Sirve para ver la jerarquía o árbol de subdirectorios, bajando los niveles que le indiquemos.

Sintaxis: tree [-opciones] [argumento]

Opciones:

-L: Indicamos el número de niveles que queremos mostrar.

Argumento: Directorio desde el que queremos visualizar el árbol. Si no indicamos argumentos nos mostrará el árbol desde la carpeta actual.

Ampliación: Lista el árbol de nuestra carpeta personal estando nosotros en el raíz solo tres niveles.
tree -L 3

4 Redireccionamiento

- La salida estándar en el sistema Operativo GNU/Linux es la pantalla
- La entrada estándar en el sistema Operativo GNU/Linux es el teclado
- Linux permite redireccionar la salida estándar, la entrada estándar y/o las salidas de errores estándar.
- Las formas más comunes de redireccionar son:
 - >
 - Redirecciona la salida de un comando hacia un archivo.
 - Si el archivo ya existe, lo borra y lo crea uno nuevo y si no existe lo crea.
 - Ejemplo;
 - `ls -ltr > listado.txt`
 - >>
 - Redirecciona también la salida estándar.
 - Si el fichero no existe, lo crea, pero si el fichero existe, añade la nueva información al final del fichero ya existente.
 - Ejemplo;
 - `df -h >> listado.txt`
 - <
 - La entrada estándar de un comando se lee desde un fichero.
 - Ejemplo;

- `grep Messi < futbol.txt`

5 Tuberías o pipelines

- Permite redireccionar *la salida de un comando como la entrada de otro comando*.
- El símbolo utilizado es `|` (*Alt Gr + 1*)
- Permite introducir varios comandos al mismo tiempo, enviando el resultado de uno al otro.
- Ejemplo;
 - `ls -ltr | more`
 - *Pagina el listado de un directorio*
 - `ls -ltr | grep futbol`
 - *Muestra si existe la cadena futbol dentro del listado de un directorio*
 - `ls -ltr | grep futbol | grep txt`
 - *Muestra si existe la cadena futbol y la cadena txt en la misma línea, dentro del listado de un directorio*
 - `cat futbol.txt | sort`
 - *Ordena alfabéticamente el contenido de un fichero*
 - `cat futbol.txt | sort | cut -d " " -f 1`
 - *Ordena alfabéticamente el contenido de un fichero mostrando sólo la primera columna antes del primer espacio*
 - `cat futbol.txt | sort | cut -d " " -f 1 | head -1`
 - *Ordena alfabéticamente el contenido de un fichero mostrando sólo la primera fila de la primera columna antes del primer espacio*
 - `cat futbol.txt | sort | cut -d " " -f 1 | head -1 | wc -c`
 - *Ordena alfabéticamente el contenido de un fichero, contando los caracteres de la primera fila de la primera columna antes del primer espacio*

6 Ejecución de un comando en segundo plano

- El símbolo utilizado es `&`
- Permite ejecutar un comando en segundo plano y poder seguir trabajando con la consola o terminal.
- Ejemplo;
 - `firefox &`
 - *Ejecuta el navegador firefox en segundo*

7 Ejecución de varios comandos a la vez

- Permite ejecutar varios comandos en la misma línea de forma simultánea
- El carácter separador de los comandos es &&
- Ejemplo;
 - `ls -ltr && cat futbol.txt && df -h`

1.7 REDIRECCIÓN DE LA ENTRADA Y SALIDA DE LOS COMANDOS

1.7.1 CONCEPTOS BÁSICOS

Antes de pasar a la parte divertida debemos manejar tres términos fundamentales en Linux: entrada estándar, salida estándar y error estándar.

La **entrada estándar** representa los datos que necesita una aplicación para funcionar, como por ejemplo un archivo de datos o información ingresada desde la terminal y es representado en la terminal como el **tipo 0**.

La **salida estándar** es la vía que utilizan las aplicaciones para mostrarte información, allí podemos ver el progreso o simplemente los mensajes que la aplicación quiera darte en determinado momento y es representado en la terminal como el **tipo 1**.

El **error estándar** es la forma en que los programas te informan sobre los problemas que pueden encontrarse al momento de la ejecución y es representado en la terminal como el **tipo 2**.

Todos estos tipos son representados físicamente como archivos en el sistema, pues como ya deben saber, todo en Linux son archivos.

1.7.2 ENTONCES, ¿QUÉ ES UNA REDIRECCIÓN?

Una redirección consiste en **trasladar la información de un tipo a otro**, por ejemplo de la salida estándar a la entrada estándar o del error estándar a la salida estándar. Esto lo logramos usando el símbolo `>`.

Por ejemplo, para redireccionar la salida de un comando y volcarla a un archivo bastaría con ejecutar:

```
$ ls -la ~ > archivo.txt
```

Sin embargo, cada vez que ejecutemos ese comando el contenido de *archivo.txt* será reemplazado por la salida del comando `LS`. Si queremos agregar la salida del comando al archivo, en lugar de reemplazarla, entonces ejecutamos:

```
$ ls -la ~ >> archivo.txt
```

Lo interesante es que, además de la salida estándar, también podemos redireccionar el error estándar y la entrada estándar. Si queremos forzar a que un programa nos imprima en pantalla los errores que consiga durante su ejecución podemos redireccionar el error estándar hacia la salida estándar. Eso lo logramos ejecutando:

```
$ programa 2>&1
```

¿Recuerdan que más arriba les explicaba que Linux identifica a cada tipo con un número? Bueno, aquí es donde esos números cobran sentido. El tipo **2** es el error estándar y el tipo **1** es la salida estándar. En los ejemplos previos no tuvimos la necesidad de especificar el tipo 1 porque la terminal lo asume pero pudimos expresarlos explícitamente de la siguiente manera:


```
$ ls -la ~ 1> archivo.txt
$ ls -la ~ 1>> archivo.txt
```

Podemos, por ejemplo, contar las líneas que tiene un archivo redireccionando la entrada estándar de **WC** hacia un archivo de texto. Así:

```
$ wc < archivo.txt
```

También podemos hacer algo muy común en la administración de sistemas, descartar el error estándar de un proceso. Para eso ejecutamos:

```
$ programa 2> /dev/null
```

O incluso descartar su salida estándar:

```
$ programa > /dev/null
```

En Linux, **/DEV/NULL** es un archivo especial al que se envía cualquier información que quiera ser descartada.

1.8 TUBERÍAS

Uno de los principios de la filosofía Unix consiste en tener aplicaciones muy pequeñas que hagan tareas muy puntuales y que usadas en conjunto puedan realizar tareas complejas. Entonces, siguiendo este principio debe existir una forma para que varias aplicaciones puedan interactuar entre sí, aquí es donde entran las tuberías.

Una tubería en Linux no es más que una forma práctica de **redireccionar la salida estándar de un programa** hacia la entrada estándar de otro. Esto se logra usando el símbolo **|** (pipe).

Por ejemplo, podríamos ver los procesos que están corriendo en el sistema usando **PS** y le redireccionamos la salida a **SORT** para que los ordene por PID:

```
$ ps -a | sort
```

También podemos redireccionar la salida estándar del comando **cat** y pasarla como entrada estándar del comando **WC** para contar las líneas y palabras de un archivo:

```
$ cat archivo.txt | wc
```

Tanto las redirecciones como las tuberías son conceptos fundamentales que sin duda debes manejar para empezar a sentirte cómodo con la terminal. Verás que al cabo de un tiempo no sabrás cómo pudiste vivir sin ellos.

1.11 EMPAQUETAR FICHEROS Y DIRECTORIOS

En este apartado veremos cómo **comprimir y empaquetar archivos o directorios, listar el contenido de una carpeta comprimida y extraer los diferentes archivos o carpetas contenidas en ella.**

Para ello trabajaremos con los diferentes formatos de compresión (o al menos los más comunes) con los que te puedes encontrar en prácticamente cualquier distribución GNU/Linux de forma nativa, tales como **zip**, **tar**, **gz**, **bz2**, **tar.gz** y **tar.bz2**, los diferentes formatos de compresión desde la línea de comandos.

1.11.1 TAR

Tar realiza operaciones de empaquetado y desempaquetado de archivos, que no compresión.

Para ello, el formato más utilizado en GNU/Linux es TAR, una utilidad que permite empaquetar varios archivos o directorios dentro de un solo paquete en formato con la extensión **.TAR**.

1.11.1.1 #tar -cvf

Podemos suponer que queremos empaquetar dos archivos, que denominaré **ARCHIVO1** y **ARCHIVO2**, dentro de un paquete **.TAR** que podemos denominar **PAQUETE**. Para empaquetar los diferentes archivos uno a uno, puedes utilizar el siguiente comando:

```
$ tar -cvf paquete.tar archivo1 archivo2
```

Cuando se trata de empaquetar varios archivos, lo más práctico es ubicarlos todos dentro de una carpeta y empaquetar directamente la directorio. Para ello puede utilizar el mismo comando anterior pero sustituyendo los diferentes archivos con el nombre del directorio, que en este ejemplo podemos denominar **DIRECTORIO**.

```
$ tar -cvf paquete.tar directorio
```

1.11.1.2 #tar -tvf

Una vez tienes un paquete **.TAR**, una de las cosas que podemos necesitar es conocer el contenido que hay dentro. Para ello debemos utilizar el comando que tienes a continuación, que listará, de forma recursiva (esto es, recorriendo todos los niveles) los diferentes archivos o directorios uno a uno.

```
$ tar -tvf paquete.tar
```

1.11.1.3 #tar -xvf

Una vez listado el contenido del paquete **.TAR** que hemos creado, lo siguiente es desempaquetar los diferentes archivos o directorios contenidos dentro. Para ello, debes tener en cuenta que si mantienes archivos o directorios con el mismo nombre fuera del paquete (en la misma ubicación) estos serán reemplazados por los nuevos, por lo que conviene que te asegures bien antes.

```
$ tar -xvf paquete.tar
```

1.11.2 GZIP Y BZIP2

En este apartado veremos los comandos básicos con los que podrás comprimir y descomprimir cualquier archivo en Linux.

En este sentido es justo señalar que, en entornos Unix, se separa lo que es el concepto de compresión de lo que es el hecho de empaquetar varios archivos en un único archivador (obviamente puedes utilizar formatos con los que combinar las dos acciones, como verás un poco más adelante).

Para la compresión pura de un archivo, en Linux se utilizan mucho el formato GZIP, todo un clásico en entornos Unix, y el formato BZIP2, algo más moderno y con un mayor índice de compresión.

Antes que nada, ten en cuenta que tenemos que situarnos en la carpeta dónde se encuentran los archivos que queremos comprimir. Para poner un ejemplo, supongamos que están en la carpeta **DOCUMENTOS** dentro de la **HOME**.

```
$ cd Documentos
```

1.11.2.1 #gzip

Gzip es una utilidad de GNU/Linux integrada en el propio sistema operativo y que permite la compresión o descompresión de archivos en formato **GZ**.

Como apunte importante, gzip es una herramienta de compresión pura, es decir, solo comprime archivos individuales (no directorios), y, a diferencia de formatos más populares como rar o zip, no permite empaquetar varios archivos dentro del mismo.

A continuación tienes los principales comandos que puedes utilizar para comprimir y descomprimir un archivo utilizando **GZIP**.

La sentencia que tienes a continuación es la que te servirá para comprimir cualquier archivo en Gzip, lo que dará como resultado un paquete comprimido con la extensión **.GZ**.

```
$ gzip -9 archivo
```

Para gestionar cualquier archivo comprimido en gzip tienes varias opciones. Si quieres conocer los principales parámetros del archivo, véase el ratio de compresión o el nombre del archivo una vez descomprimido, puedes utilizar la siguiente sentencia:

```
$ gzip -l archivo.gz
```

Y finalmente, para descomprimir un archivo comprimido en gzip (que podrás reconocer porqué termina con la extensión **.GZ**), puedes utilizar la siguiente sentencia:

```
$ gzip -d fichero.gz
```

1.11.2.2 #bzip2

La utilidad **bzip2** también forma parte del sistema en GNU/Linux y es muy similar a gzip, lo único que en este caso sirve para comprimir o descomprimir archivos utilizando el formato **BZ2**.

Al igual que ocurre con gzip, bzip2 solo permite la compresión de archivos individuales, ya que se trata de una herramienta puramente de compresión.

Para comprimir un archivo cualquiera, que en este ejemplo voy a denominar **ARCHIVO**, en bzip2, la sentencia que debes utilizar es la que tienes a continuación.

```
$ bzip2 -z archivo
```

Una vez tienes el archivo comprimido en la extensión **BZ2**, que en este caso se llama *archivo.bz2*, puedes hacer un test de la integridad del mismo con el comando que tienes debajo. Si todo está OK, no habrá ningún output.

```
$ bzip2 -t archivo.bz2
```

Siguiendo con el ejemplo anterior, para descomprimir el archivo *archivo.bz2* puedes utilizar la sentencia de debajo.

```
$ bzip2 -d archivo.bz2
```

1.11.3 TAR.GZ Y TAR.BZ2

Llegados este punto, creo que es el momento de tratar con las operaciones que combinan compresión y descompresión de archivos, que al fin y al cabo son las más comunes en otro tipo de entornos como Windows (de hecho, así es como trabajan los formatos de compresión más extendidos, léase Zip y Rar, entre otros).

Si cuando hemos hablado de Gzip y Bzip2 hemos visto que se trata de dos utilidades puramente de compresión, pues bien, puedes combinar ambas utilidades con Tar, dando lugar a archivos con las extensiones **.tar.gz** y **.tar.bz2**, que seguramente te resultaran muy familiares en Linux.

Esto da lugar a la posibilidad de comprimir múltiples archivos dentro de un mismo archivador, al igual que puedes hacer con otros formatos más populares. A continuación se resumen los comandos más importantes que debes conocer en este sentido:

1.11.3.1 #tar -czvf

Para comprimir varios archivos (a modo de ejemplo vamos a llamarlos ARCHIVO1 y archivo2) dentro de un paquete comprimido **tar.gz**, puedes utilizar el siguiente comando. Como verás, es el mismo que utilizamos para empaquetar archivos en tar, con el añadido del parámetro **-C**, que representa la compresión en Gzip.

```
tar -czvf paquete.tar.gz archivo1 archivo2
```

También nos puede interesar comprimir un directorio entero, con todo su contenido dentro. En este caso, la sentencia sería la misma pero sustituyendo los diferentes archivos por el nombre del directorio.

```
tar -czvf paquete.tar.gz directorio
```

1.11.3.2 #tar -xzvf

Para descomprimir el paquete y desempaquetar su contenido, seguimos con la misma estructura, pero en este caso sustituimos el parámetro **-C**, de compresión, por el parámetro **-X**, de descompresión. A continuación tienes la sentencia completa.

```
tar -xzvf paquete.tar.gz
```

1.11.3.3 #tar -cjvf

Para empaquetar y comprimir una serie de archivos, en este ejemplo utilizaremos los archivos denominados ARCHIVO1 y ARCHIVO2 en un paquete **.tar.bz2** te puedes valer de la siguiente sentencia, que si te fijas, es igual que en el caso de los archivos **.tar.gz** pero cambiando la **-Z** por la **-J**.

```
tar -cjvf paquete.tar.bz2 archivo1 archivo2
```

1.11.3.4 #tar -xjvf

Para descomprimir un paquete **.tar.bz2** y desempaquetar su contenido, seguimos con la misma estructura, pero en este caso sustituimos nuevamente el parámetro **-C**, de compresión, por el parámetro **-X**, de descompresión. A continuación tienes la sentencia completa.

```
tar -xjvf paquete.tar.bz2
```

◀ Despedida ▶

Email

bienvenidosaez@gmail.com

Instagram

@bienvenidosaez

Youtube

youtube.com/bienvenidosaez